# Final Project Proposal: Autonomous ML Training Agent

**CS194: Software Project**

Stanford University, Spring 2026

*Date: April 12, 2026 | Version: 1.0 (Final for Submission)*

*Members: Sid Potti, Matthew Torre, Ron Polonsky, Angel Raychev*

## Overview

The Autonomous ML Training Agent is an end-to-end system that takes a plain-English prompt (e.g., "build a model that classifies handwritten digits") and a strict student budget cap (e.g., $50), then automatically handles data discovery/generation, model architecture selection, hyperparameter tuning, and training orchestration on Tinker's distributed GPU infrastructure. The system enforces hard budget guardrails and terminates runs before exceeding allocated costs, eliminating the financial anxiety that paralyzes student ML projects.

*Core Innovation: An autonomous ML "co-pilot" that collapses weeks of data wrangling, model selection, and infrastructure setup into a single API call to make custom model training accessible to non-ML researchers, students, and bootstrapped teams.*

## The Problem & Market Opportunity

### Why We're Solving This

Current ML project workflows are broken for students and non-specialists:

- Data wrangling dominates. Even with a clear PyTorch architecture in mind, finding, cleaning, and formatting data consumes 80% of project time. Our agent automates discovery (HuggingFace, synthetic generation, intelligent scraping) and standardization.
- Architecture paralysis is real. Fine-tune or train from scratch? LoRA or full weights? 8B or 235B? Without domain expertise, teams waste weeks on decisions. Our Decision Engine chooses the right strategy based on task and budget.
- Cloud costs are terrifying. A forgotten GPU run = $400+ charges. Students do not have financial buffer for infrastructure mistakes. We provide a financial bodyguard with a hard kill-switch tied to real-time billing.
- Hyperparameter tuning is tedious. Manual iteration (tweak → wait → check loss → repeat) takes days. Our AutoResearch loop runs dozens of controlled experiments in hours, proposing, testing, and adopting improvements like a senior ML engineer.

### Market Size & Opportunity

**SOM (Serviceable Obtainable Market):**
Academic researchers (bio, econ, social science labs) who need custom ML models for papers but lack ML infrastructure knowledge. Stanford has 500+ faculty-led research groups; even 5% adoption = significant user base.

**SAM (Serviceable Available Market):**
As the agent's AutoResearch loop stabilizes, the market expands to bootstrapped startups, indie hackers, and small agencies. Roughly 5–10 million developers globally want to train custom AI models but cannot afford $150k+/year MLOps engineers.

**TAM (Total Addressable Market):**
The ultimate vision: the global software engineering workforce (~30 million developers). Only 2 million are specialized in ML. Our agent bridges this gap, giving the remaining 26+ million standard developers autonomous infrastructure to train and deploy custom models without specialized teams.

## User Segments & Value Proposition

### Primary Segments

- Student ML Builders: CS students needing quick, budget-conscious training for class projects or portfolios.

- Non-ML Academic Researchers: PhD students in bio, econ, linguistics, etc., who need custom classifiers or fine-tuned models for their thesis data but lack infrastructure expertise.
- Lightweight Prototyping (Research & Enterprise): Teams validating ML feasibility in days, not weeks.

### Secondary Segments

- Bootstrapped Founders / Small Startups: Teams needing custom models for a product but unable to hire dedicated ML talent or manage GPU clusters.
- General-Purpose AI Experimenters: Hackers, indie researchers, and hobbyists exploring new model architectures and training strategies.

### Core Value Proposition

*A fully autonomous AI "co-pilot" that transforms a plain-English idea and a hard budget cap into a deployed, trained model by automatically searching, curating, and/or generating data; choosing the right training strategy (LoRA, fine-tune, or pre-train); running an intelligent AutoResearch loop; and enforcing strict cost guardrails. Ship useful models without touching infrastructure or blowing up cloud bills.*

## Functional Requirements & System Features

The system is organized around five core features, orchestrated by a central Manager agent:

### Feature 0: The Manager

Central orchestrator that receives the user's training prompt and budget, then sequences the downstream features (Data Generator → Decision Engine → AutoResearch Loop) while monitoring the Cost Manager for budget violations. Logs all decisions and maintains audit trail.

### Feature 1: The Data Generator

Intelligently discovers or creates training data in three modes:

#### Mode A (Existing Data): User provides messy data (CSV, JSON, images, etc.)

- Detect format and data type
- Normalize, clean, and filter
- Augment with synthetic data if needed

#### Mode B (HuggingFace): No user data; suitable dataset exists on HuggingFace Hub

- Search HuggingFace database for candidates
- Rank by relevance and select best match
- Download and validate

#### Mode C (Generation): No user data; HuggingFace search fails

- Determine data format, input/output schema
- Intelligently scrape web OR generate synthetic data using an LLM-based teacher
- Morph collected data into standardized format

### Feature 2: The Decision Engine

Analyzes the user's task and budget to recommend a training strategy:

#### Case A (Fine-Tune): Suitable when base models exist on HuggingFace

- Grab pre-trained base model from HuggingFace
- Run LoRA-based fine-tuning script on Tinker

**Case B (Pre-Train): Suitable for novel architectures or custom objectives**

- Write custom model.py and train.py from scratch
- Hand off to AutoResearch loop for continuous improvement

## Feature 3: AutoResearch Agentic Loop

The core innovation: autonomous hyperparameter and architectural tuning. After receiving a baseline model from the Decision Engine, the loop continuously proposes, tests, and adopts improvements, operating like a fast, budget-conscious ML engineer running dozens of controlled experiments.

**Loop Lifecycle:**

1. Propose: Generate a code or config diff encoding a single hypothesis (e.g., "increase context length to 1,024").
2. Run: Schedule a constrained experiment on Tinker (e.g., 5-minute budgeted run) with standardized logging.
3. Evaluate: Score the run using the Evaluator sub-feature.
4. Decide: Merge winning diffs; revert losing ones. Maintain a research diary of all iterations.

**Search Strategy & Safeguards:**

- Uses simple but effective strategies: random search over bounded ranges, local perturbations around current best, heuristic "playbooks" from prior runs.
- Early stopping: If a run shows catastrophic degradation (exploding loss, NaNs, accuracy collapse) within the first few steps, it is terminated and budget reallocated.
- All edits tracked as discrete patches to training script or config, mirroring Karpathy's autoresearch design.
- Human-readable research diary logs iteration details: what was tried, cost, metric movement.

**Evaluator Sub-Feature:**
Manages the evaluation surface for every iteration. Responsibilities:

- Create or select the right eval suite for the task (hold-out validation split, synthetic stress tests, LLM-graded benchmarks).
- Run evals automatically after each experiment.
- Return a single scalar score plus natural-language critique back to the loop.
- Flag regressions (accuracy drop, calibration worsening, poor cost-per-gain) and trigger reverts.
- Adapt eval set over time by adding harder examples when systematic weaknesses are detected.

## Feature 4: The Cost Manager

- Continuously polls Tinker billing API.
- If cumulative spend exceeds user budget, saves model state_dict and nukes the GPU instance immediately.
- Returns final model weights and a cost breakdown to the user.

## Feature 5: Observability

- Real-time logging of all agent decisions, training metrics, and budget tracking.
- Structured logs (JSON) for programmatic access.
- (Stretch) Lightweight web dashboard to monitor in-flight training runs and AutoResearch iterations.

## Metrics for Success

### Objective Function

Minimize time-to-trained-model for students and researchers building custom classifiers, while maintaining cost ≤ user-specified budget cap, with zero human infrastructure intervention required.

### Internal (Development) Metrics

| Metric | Target by June | Owner |
|---|---|---|
| **% tasks completed end-to-end without human intervention** | 80% for baseline classification tasks | Team |
| **Time from prompt → trainable model** | < 20 min (classification), < 45 min (fine-tuning) | Ron + Angel (DataGen) |
| **Cost prediction accuracy** | Within 10% of actual spend | Sid (Orchestrator) |
| **AutoResearch iterations per hour** | 3–5 meaningful tests per budget hour | Matthew (AutoResearch) |
| **Baseline vs. improved model lift** | AutoResearch beats baseline by ≥ 3% | Matthew (Evaluator) |

### Adoption Metrics (If Live User Testing)

- # of unique users (grad students, researchers) who test the system: target ≥ 3 by demo day.
- # of models trained end-to-end: target ≥ 5.
- Average cost per model: target < $15 per student project (within typical ML budget).
- User sentiment: "Would use again" (from feedback survey): target ≥ 70%.
- Data Generator success rate (at least one of three modes succeeds for any prompt): target ≥ 90%.

## Timeline & Milestones

### 10-Week Development Plan

| Phase | Timeline | Deliverables | Owner(s) |
|---|---|---|---|
| **MVP Build** | Apr 15 – May 5 (Weeks 1–3) | Feature 0 (Manager), Feature 1 (basic DataGen), Feature 2 (fine-tune path), Feature 4 (cost kill-switch) | Team (Ron/Angel/Sid) |
| **AutoResearch Build** | May 6 – May 26 (Weeks 4–6) | Feature 3 (loop + search strategy), Evaluator sub-feature, early testing | Matthew (lead) |
| **Integration & Polish** | May 27 – Jun 9 (Weeks 7–8) | End-to-end system test, Feature 5 (observability MVP), bug fixes, documentation | Team |

| | | | |
|---|---|---|---|
| **Demo & Final Polish** | Jun 10 – Jun 20 (Weeks 9–10) | Finalize demo script, create case study/video walkthrough, prepare presentation | Team |

### Key Milestones

- May 5: MVP complete. All Features 0–2 and Feature 4 (Cost Manager) functional. Can train a basic classifier end-to-end on Tinker.
- May 26: AutoResearch loop stable. Feature 3 fully integrated; Evaluator working. System can improve a baseline model by ≥ 3% across test tasks.
- Jun 9: System hardened. All features integrated. Pass 3 diverse test scenarios (classification, fine-tuning, custom architecture).
- Jun 15–20: Demo day ready. Live demo running on Tinker. One 10-minute walkthrough of entire pipeline.

## Design & UX Considerations

### User Interface & Input/Output Design

The system is CLI/API-first, designed for programmatic use and straightforward terminal interaction. No web UI for MVP; observability is log-based.

**Input Interface:**

- YAML config file or command-line args: prompt, budget, task type, optional data path.
- Example: agent.py —prompt "classify sentiment in movie reviews" —budget 50 —task classification

**Output Interface (Real-Time Logs):**

Human-readable CLI output with structured logging (JSON for programmatic access):

```
[Agent] Task: Classify movie review sentiment
[Agent] Budget: $50.00 | Spent: $0.00
[Manager] → Initializing system...
[DataGen] Searching HuggingFace...
[DataGen] ✓ Found 'movie_reviews' dataset (50K examples)
[DataGen] Downloading and validating...
[DecisionEngine] Task Analysis: Sentiment classification (NLP)
[DecisionEngine] Recommendation: Fine-tune BERT (cost: $12, time: 8m)
[AutoResearch] Iteration 1/8: Testing LoRA rank=16...
  → Loss: 0.35 → Val F1: 0.89 ✓ [KEPT]
[AutoResearch] Iteration 2/8: Testing learning_rate=2e-5...
  → Loss: 0.33 → Val F1: 0.91 ✓ [KEPT]
[CostManager] Spend: $9.20 / $50.00 (18% used)
[Agent] Training complete!
[Agent] Final Model: F1 = 0.93 | Cost: $11.50
[Agent] Saved to: ./models/sentiment_classifier.pt
```

### Design Principles

- Clarity over brevity: Users need to understand what the agent is doing and why at each step.
- Financial transparency: Cost must be visible in real time. No surprises.
- Debuggability: All edits, experiments, and decisions are logged and reproducible.
- Minimal input friction: Users provide a sentence, a number, optional data. The system figures out the rest.

### Stretch Goal: Observability Dashboard

- Simple web dashboard (if time permits in Week 8): Monitor in-flight runs, view real-time metrics, inspect AutoResearch diary.
- Backend: Expose system state via REST API; frontend: simple React dashboard.

- Not required for June demo but would significantly enhance perceived polish.

## Technical Considerations & Architecture

### System Architecture Overview

- Manager (orchestrator): Python agent that sequences downstream components and manages state.
- Data Generator: Independent module (Python); interfaces with HuggingFace Hub API, web scrapers, and synthetic LLM-based generation.
- Decision Engine: Rule-based module (Python) that evaluates task requirements and budget to recommend fine-tune vs. pre-train.
- AutoResearch Loop: Agentic loop (Python) built atop Karpathy's autoresearch scaffold; maintains training script as mutable code, experiments tracked as diffs.
- Cost Manager: Polling agent (Python) that monitors Tinker billing API and enforces hard budget kills.
- Tinker Integration: All GPU training runs submitted to Tinker; we consume their job API and billing API.

### Technical Constraints & Risks

#### Data Generator (Feature 1)

- Risk: Web scraping is fragile. Websites change, rate limits apply, robustness is hard.
- Mitigation: Prioritize HuggingFace search + synthetic generation. Web scraping is fallback only. Implement robust error handling and user-friendly failure modes.
- Data quality: What constitutes "good enough" data? For MVP, we require: accurate labels, representativeness (dataset matches deployment distribution), and completeness (no missing values). This is validated heuristically (sample inspection) and through training curves (if loss plateaus early, data quality is suspect).

#### AutoResearch Loop (Feature 3)

- Risk: Diff tracking + merging can break the training script if concurrent edits conflict.
- Mitigation: Ensure only one agent thread is modifying the script at a time. Use version control (Git) internally; track all diffs. Revert on any error.
- Risk: Evaluator LLM grading (if used) has latency and cost overhead.
- Mitigation: Use lightweight evals for MVP (validation accuracy, F1, loss curves). LLM grading is optional and budgeted separately.

#### Cost Manager (Feature 4)

- Risk: Tinker billing API latency. If there's a delay between spending and billing visibility, we could overspend.
- Mitigation: Poll Tinker API every 30 seconds. Maintain a conservative buffer (e.g., stop at 90% of budget, not 100%). Clarify Tinker's billing SLA with their team.
- Risk: If a run exceeds budget, we kill the instance and lose state.
- Mitigation: Save model state_dict frequently (every 5–10 minutes). Return the best checkpoint found to the user.

#### Tinker API Dependencies

- We rely on two Tinker APIs: Job Submission (launch training runs) and Billing (monitor spend).
- Outstanding: Sid is tracking down updated Tinker docs. This is a blocker for Weeks 1–2 work; must resolve by Apr 18.
- Rate limits: Confirm Tinker's rate limits for job submission and billing queries. Budget for backoff/retry logic.

### Key Technical Decisions

- Language: Python 3.10+ (required for modern async syntax, type hints, and ML ecosystem compatibility).
- Training framework: PyTorch (flexibility for custom architectures; matches CS124/231N course tools).
- Model sourcing: HuggingFace Transformers library (largest open-source model hub).
- AutoResearch scaffold: Adapted from Karpathy's minimal autoresearch repo. Single mutable training script + research diary (JSON log).
- State management: In-memory agent state + persistent JSON logs (research diary, cost ledger, model checkpoints on shared storage).
- LLM-powered decisions: Use Claude API (via Anthropic SDK) for prompt interpretation, data generation, and architectural suggestions. Budget per call; fallback to rule-based defaults if API quota exhausted.

## Go-to-Market Plan

### Messaging & Positioning

**Core Pitch:**
***"Train a custom ML model in 20 minutes with a single prompt. No infrastructure headaches, no surprise bills. AutoML for students and researchers."***

**Key Messages by Segment:**
**For Student Builders:** "Stop wrangling data. Start shipping. Our agent handles the busywork so you can focus on what matters."

**For Academic Researchers:** "Custom ML models for your thesis data without needing to hire a postdoc. Faster papers, better science."

**For Bootstrapped Founders:** "Deploy ML without ML engineers. Train custom models on your data, stay lean, scale faster."

### Launch Strategy & Channels

| Phase | Timeline | Channel(s) | Content & Action |
|---|---|---|---|
| **Internal Testing** | May 1–15 | Office hours, team Slack | TA feedback loop. Refine docs and demo script. |
| **Soft Launch** | May 15–31 | Stanford Slack (#cs-builders, #ml-builders), email to Bio/Econ labs | Brief pitch + GIF walkthrough. Recruit 2–3 beta users for feedback. |
| **Beta Testing** | Jun 1–9 | 1-on-1 user interviews, async feedback form | Iterate based on real user pain points. Document case studies. |
| **Public Demo** | Jun 15–20 | Demo day, portfolio, GitHub (open-source?) | Live system walkthrough. Publish case study + README. Consider open-source release. |

### Content & Artifacts to Create

- README.md: Quick-start guide, code examples, architecture diagram.
- 5-min demo walkthrough GIF/video: User prompt → final trained model in real time.
- Case study: "How bio lab trained a custom protein classifier in 45 minutes for $10." (real or simulated).
- Presentation deck (10 slides): Problem, solution, demo, market, future.
- Architecture whitepaper (2–3 pages): System design, feature specs, technical trade-offs.

## Open Questions & Assumptions

### Product & Feature Scope

- What is the minimum viable dataset size? When does the Data Generator switch from HF search to synthetic generation? (Decision rule needed by Week 2)
- How complex should the AutoResearch loop be for MVP? Start with simple local search (random perturbations) or implement Bayesian optimization from the start? (Trade-off: speed vs. sophistication)
- Should we support deployment (inference)? For MVP, assume we only train and save weights. Deployment is post-June.
- How do we handle data privacy? If a researcher uploads proprietary data, do we scrub logs? (Clarify by Week 3)

### Technical & Integration

- Tinker API blockers: Sid must confirm updated Tinker job submission + billing API docs by Apr 18. This is blocking all team work in Week 1.
- Can we use Tinker's GPU cluster for development & testing, or do we have a separate account? (Cost & quota implications)
- How do we manage LLM calls for AutoResearch proposals + Evaluator grading without exploding costs? Set per-request budgets or use cheaper models (e.g., Claude Haiku)?
- Can we run the Manager agent locally while submitting training jobs to Tinker? Or does the whole system need to live on Tinker's infrastructure?

### User & Market

- What does "success" look like? Shipping a working MVP? Acquiring 3 real users? Demonstrating ≥3% improvement via AutoResearch? (All three, but priority order?)
- How do we recruit beta testers without doing heavy sales? Leverage Stanford relationships + email outreach to known researchers.
- Is there a post-June roadmap? (e.g., deployment, multi-modal support, enterprise licensing)? For now: focus on June demo only. Discuss roadmap during final presentation.
- Should we open-source after June? Pros: community trust, adoption. Cons: competitor risk, maintenance burden. Decision: not required for MVP. Evaluate at Week 8.

## Success Criteria for June 20 Demo

1. Live system demo: User enters prompt + budget → system trains a model end-to-end on Tinker without human intervention.
2. Cost guardrails working: System respects budget cap and terminates gracefully if approaching limit.
3. AutoResearch loop delivers: Final model beats baseline by ≥ 3% via automated hyperparameter + architectural tuning.
4. Supports ≥ 3 task types: basic classification, fine-tuning LLM, custom architecture training.
5. Case study or beta user feedback: Real or realistic scenario showing tangible value.
6. Polished presentation: 10-min walkthrough, clear messaging, architectural clarity.